



Ejercicio E01: Ochos

(CAPÍTULO 1 – PRE01)

Domingo Mery 

Ciencia de la Computación
Universidad Católica – Chile
<http://domingomery.ing.uc.cl>



PREGUNTA 01: LEER Y DESDPLEGAR LA IMAGEN DE ENTRENAMIENTO

(4 puntos)

1.1 > (1 punto) Usando el comando "wget", sube al ambiente de colab el archivo que contiene la imagen de entrenamiento que está en el github del curso > pr > clases > PR01_Introduccion > datos > numbers.png

1.2 > (1 punto) Cambia el nombre del archivo a "numbers.png"

1.3 > (1 punto) Carga en la variable "I" el archivo "numbers.png"

1.4 > (1 punto) Despliega la imagen



Guardar en variable I

PREGUNTA 01: LEER Y DE

(4 puntos)

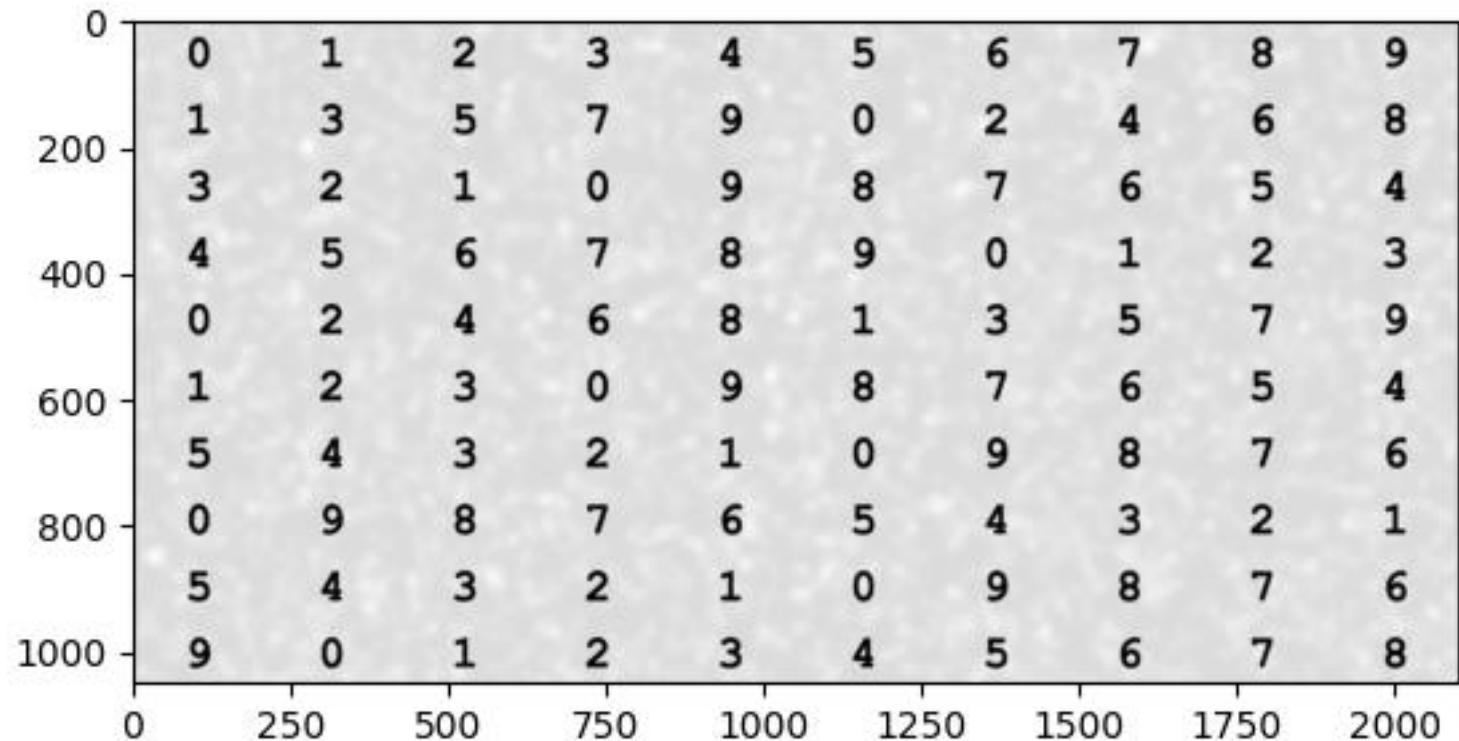
1.1 > (1 punto) Usando el comando "wget"
github del curso > pr > clases > PR01_Intr

1.2 > (1 punto) Cambia el nombre del arch

1.3 > (1 punto) Carga en la variable "I" el a

1.4 > (1 punto) Despliega la imagen

numbers.png



PREGUNTA 02: GENERARA LAS IMAGENES DE LA BASE DE DATOS

(10 puntos)

Observar que la imagen de entrenamiento tiene 100 subimágenes (una para cada uno de los 100 dígitos) que se han distribuido en 10 filas (horizontales) y 10 columnas (verticales).

2.1 > (1 punto) Crear el directorio 'img' donde se guardaran las 100 subimágenes en formato png, una por cada dígito.

2.2 > (1 punto) Imprimir el tamaño de la imagen de entrenamiento 'l'

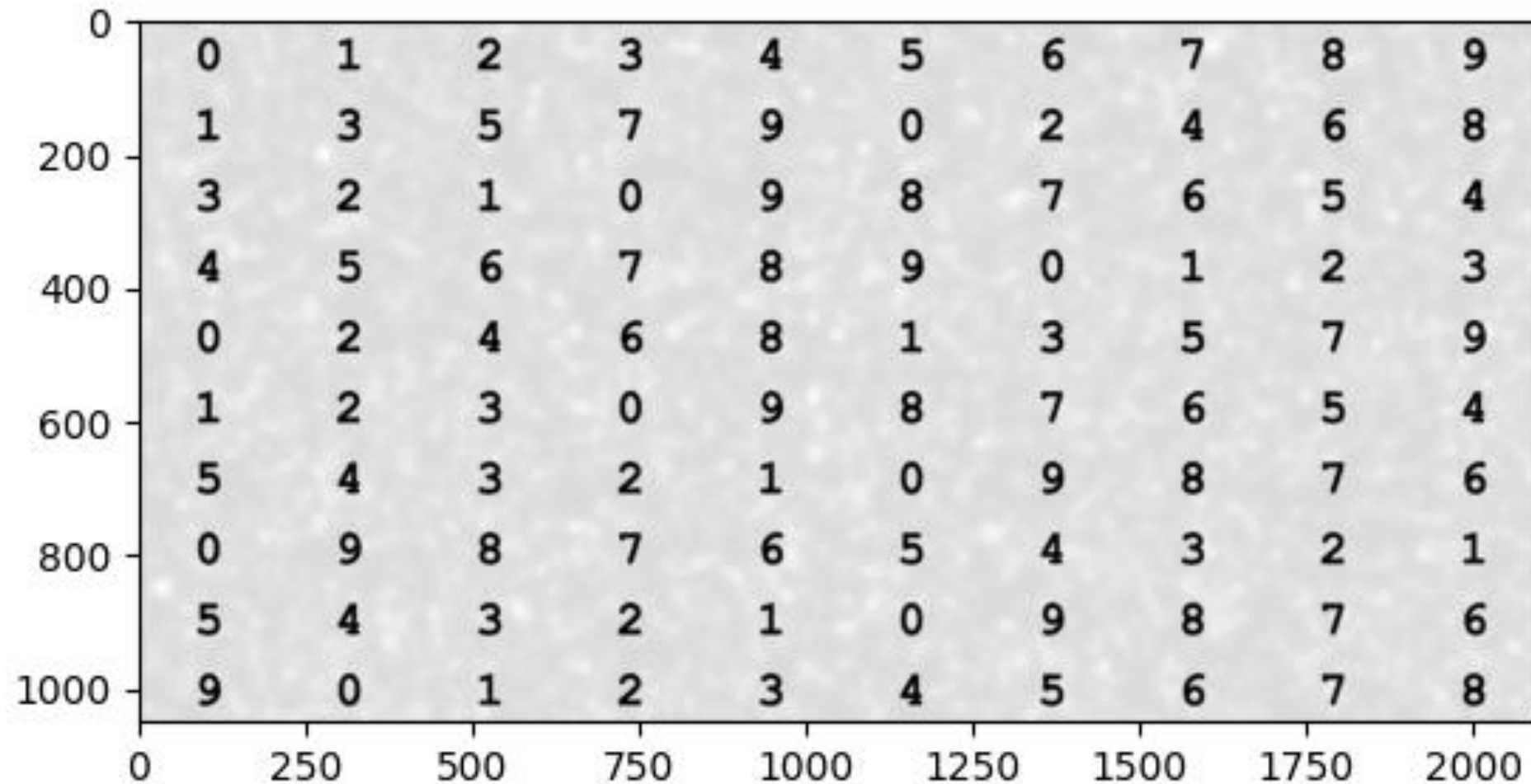
2.3 > (1 punto) Calcular el tamaño de cada subimagen (deben ser números enteros sin decimales).

2.4 > (5 puntos) Considerando el tamaño calculado en el paso anterior, genera las 100 subimágenes, una para cada dígito, leyendo fila por fila, y en cada fila de leyendo izquierda a derecha, de esta manera el dígito que esta arriba a la izquierda es la imagen '001', arriba a la derecha es la imagen '010', abajo a la izquierda es la imagen '091' y abajo a la derecha es la '100'. El nombre de cada subimagen debe ser 'img_nnn.png' donde 'nnn' va de '001' a '100'.

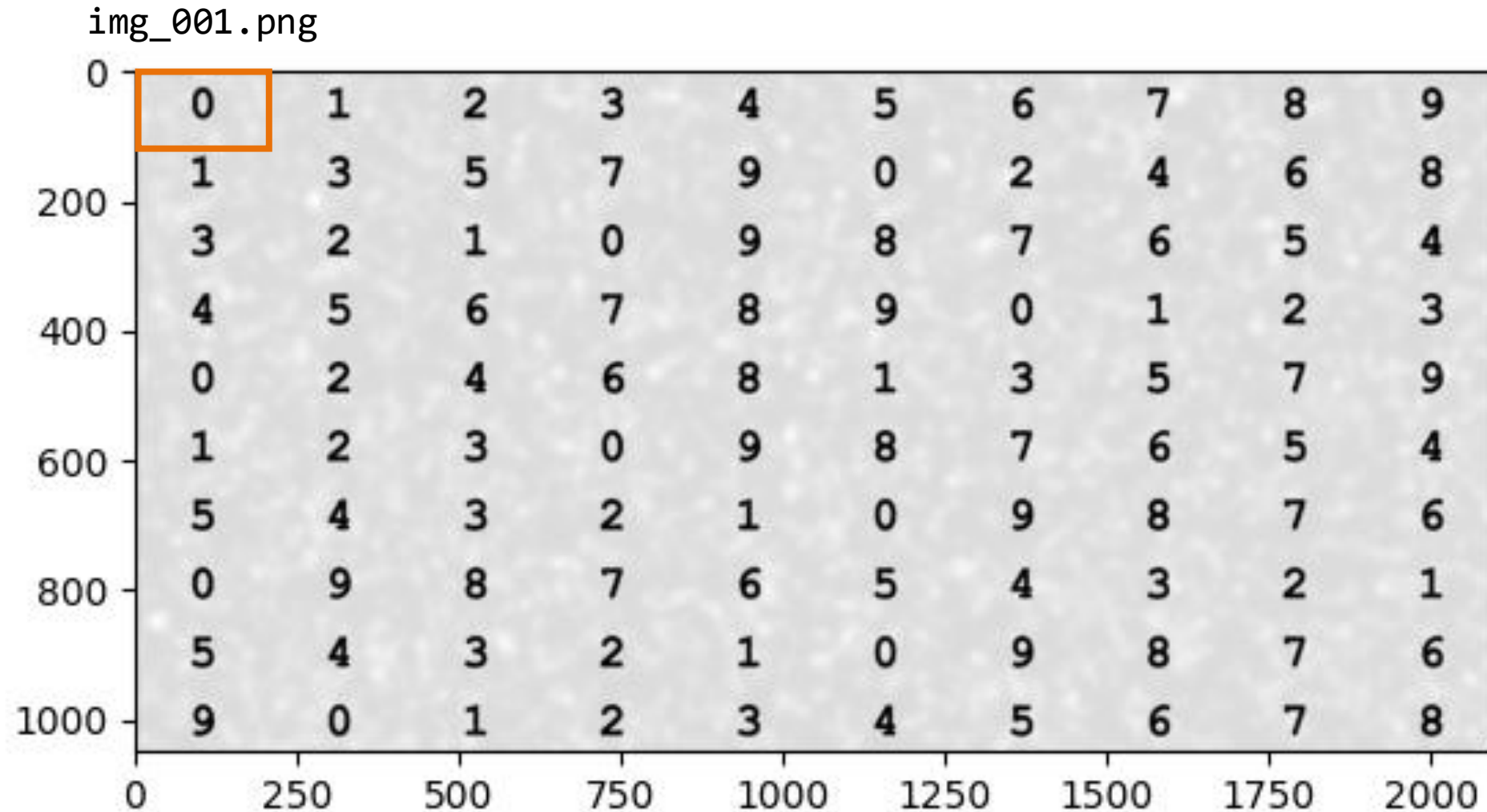
2.5 > (1 punto) Desplegar la imagen 'img/img_043.png' (debe ser un '4')

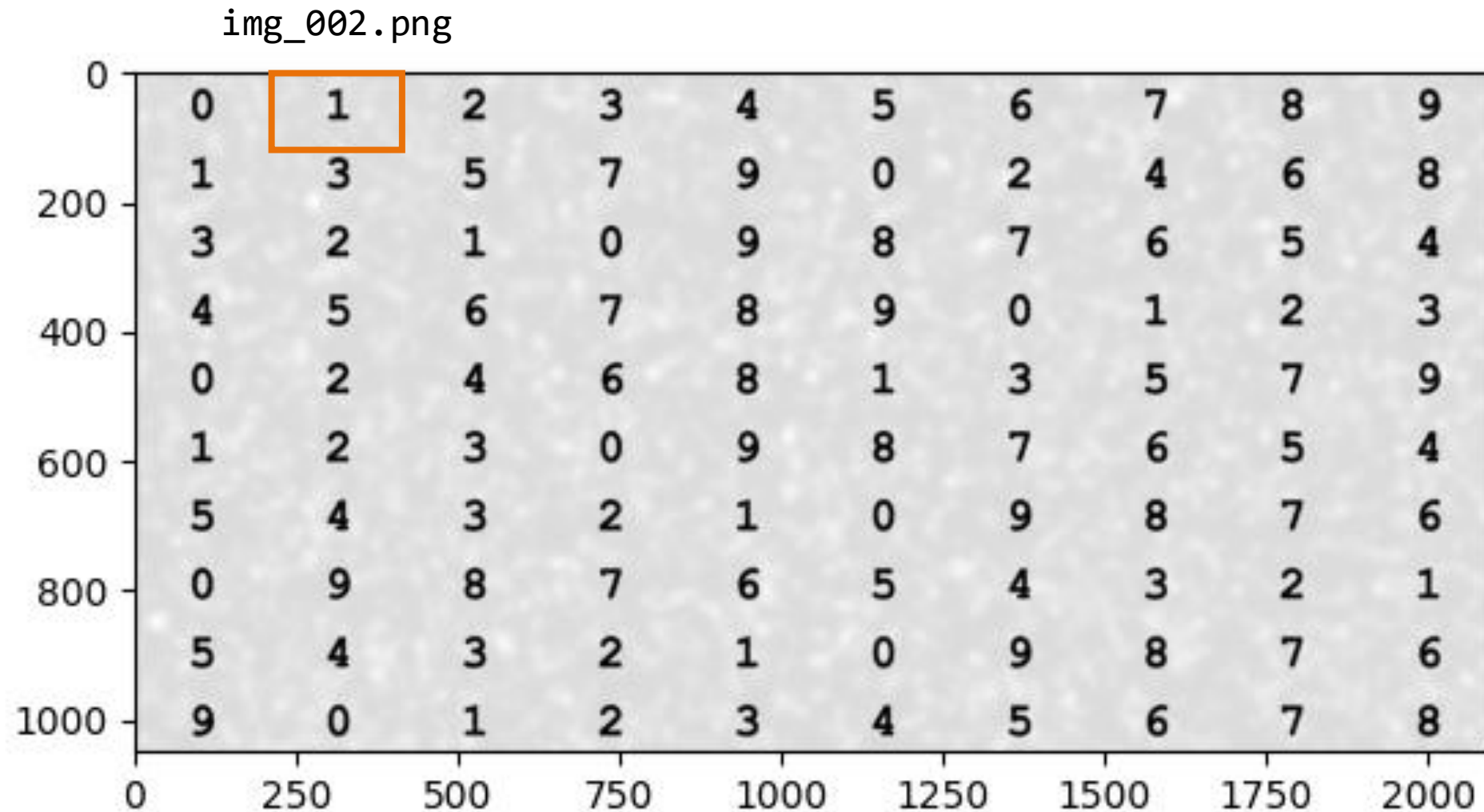
2.6 > (1 punto) Desplegar la imagen 'img/img_009.png' (debe ser un '8')

colab E01/P02

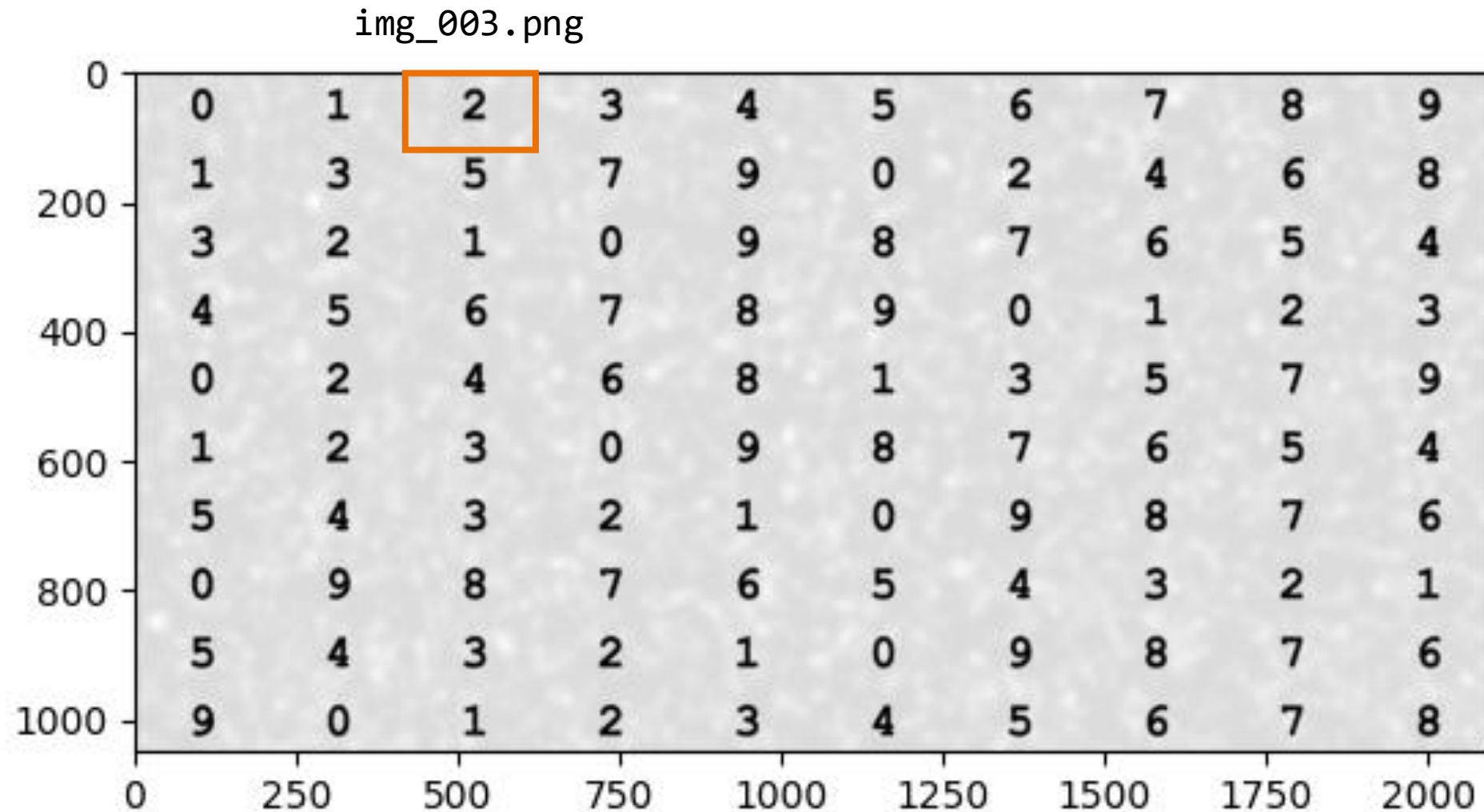


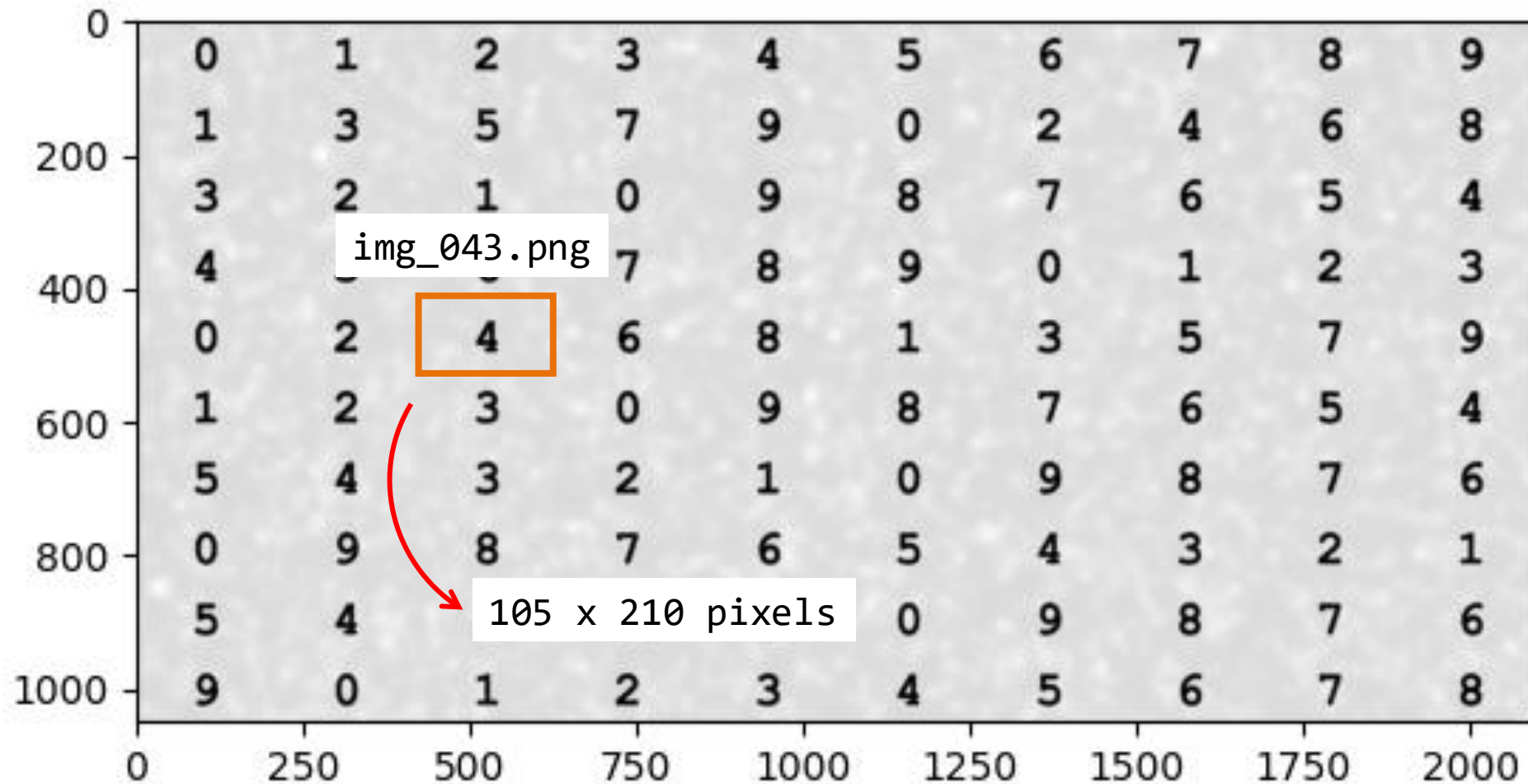
colab E01/P02



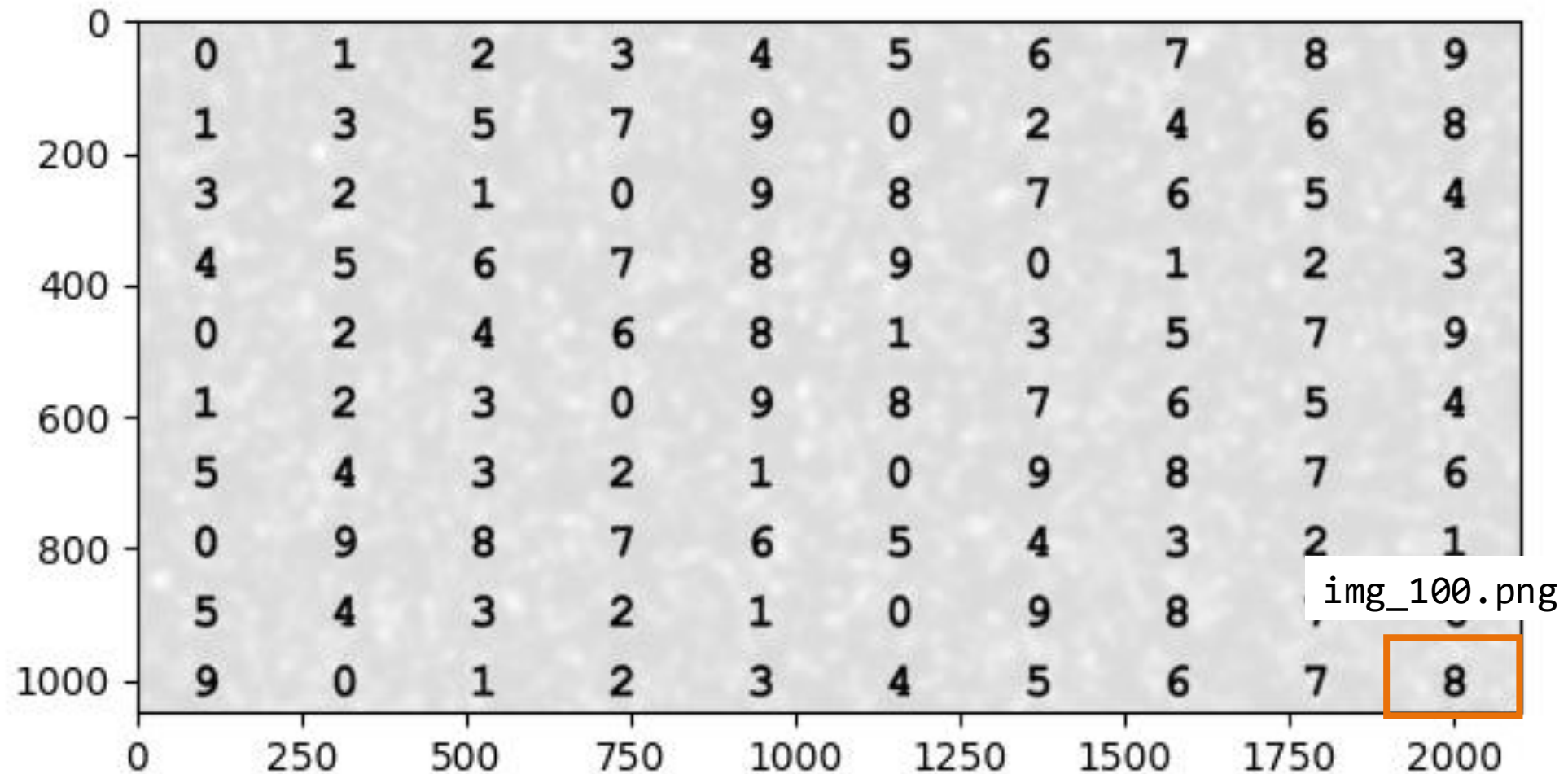


colab E01/P02





colab E01/P02



PREGUNTA 02: GENERARA LAS IMAGENES DE LA BASE DE DATOS

(10 puntos)

Observar que la imagen de entrenamiento tiene 100 subimágenes (una para cada uno de los 100 dígitos) que se han distribuido en 10 filas (horizontales) y 10 columnas (verticales).

2.1 > (1 punto) Crear el directorio 'img' donde se guardaran las 100 subimágenes en formato png, una por cada dígito.

2.2 > (1 punto) Imprimir el tamaño de la imagen de entrenamiento 'l'

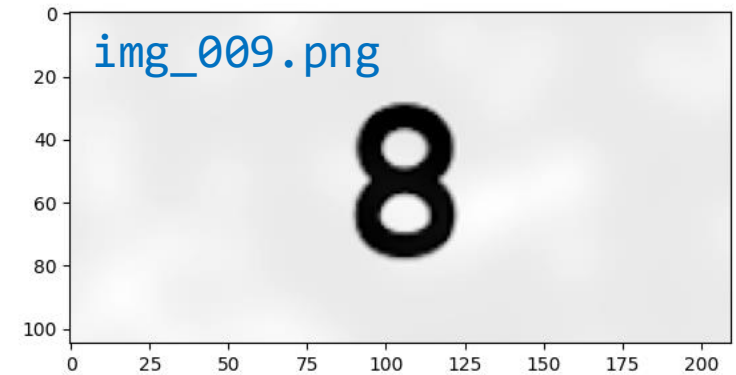
2.3 > (1 punto) Calcular el tamaño de cada subimagen (deben ser números enteros sin decimales).

2.4 > (5 puntos) Considerando el tamaño calculado en el paso anterior, genera las 100 subimágenes, una para cada dígito, leyendo fila por fila, y en cada fila de leyendo izquierda a derecha, de esta manera el dígito que esta arriba a la izquierda es la imagen '001', arriba a la derecha es la imagen '010', abajo a la izquierda es la imagen '091' y abajo a la derecha es la '100'. El nombre de cada subimagen debe ser 'img_nnn.png' donde 'nnn' va de '001' a '100'.

2.5 > (1 punto) Desplegar la imagen 'img/img_043.png' (debe ser un '4')

2.6 > (1 punto) Desplegar la imagen 'img/img_009.png' (debe ser un '8')

2.6 > (1 punto) Desplegar la imagen 'img/img_



PREGUNTA 03: SEGMENTAR LAS SUBIMAGENES

(8 puntos)

- 3.1 > (1 punto) Crear el directorio 'seg' donde se guardaran las 100 imágenes segmentadas en formato png, una por cada dígito.
- 3.2 > (5 puntos) Segmentar con un umbral adecuado cada una de las 100 subimágenes: por cada subimagen original `img_nnn.png` se debe crear una imagen `seg_nnn.png` que debe ser guardada en el directorio `seg`, para `nnn = '000', '001, ... '100'`
- 3.3 > (1 punto) Desplegar la imagen '`seg/seg_043.png`' (debe ser un '4')
- 3.4 > (1 punto) Desplegar la imagen '`seg/seg_009.png`' (debe ser un '8')

PREGUNTA 03: SEGMENTA

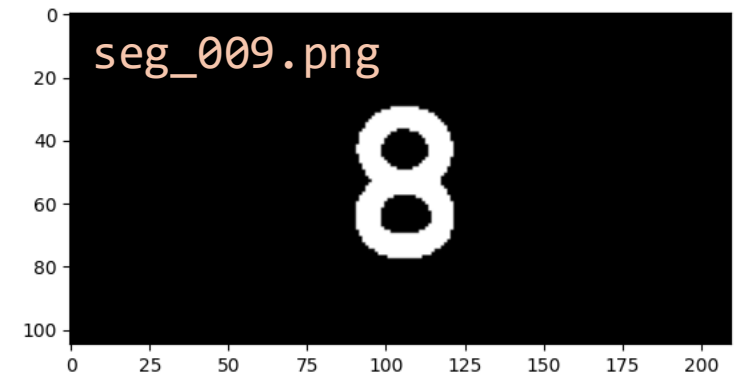
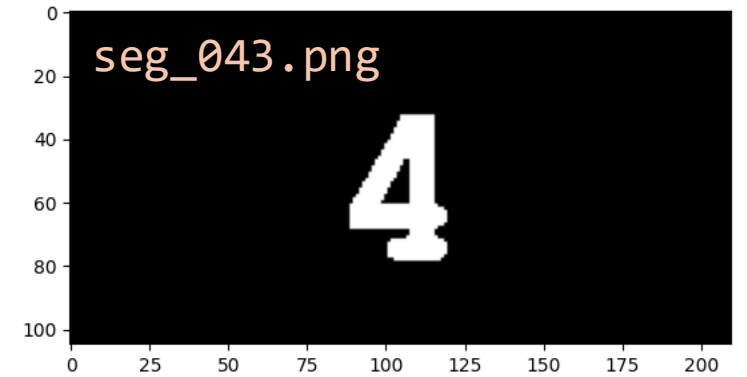
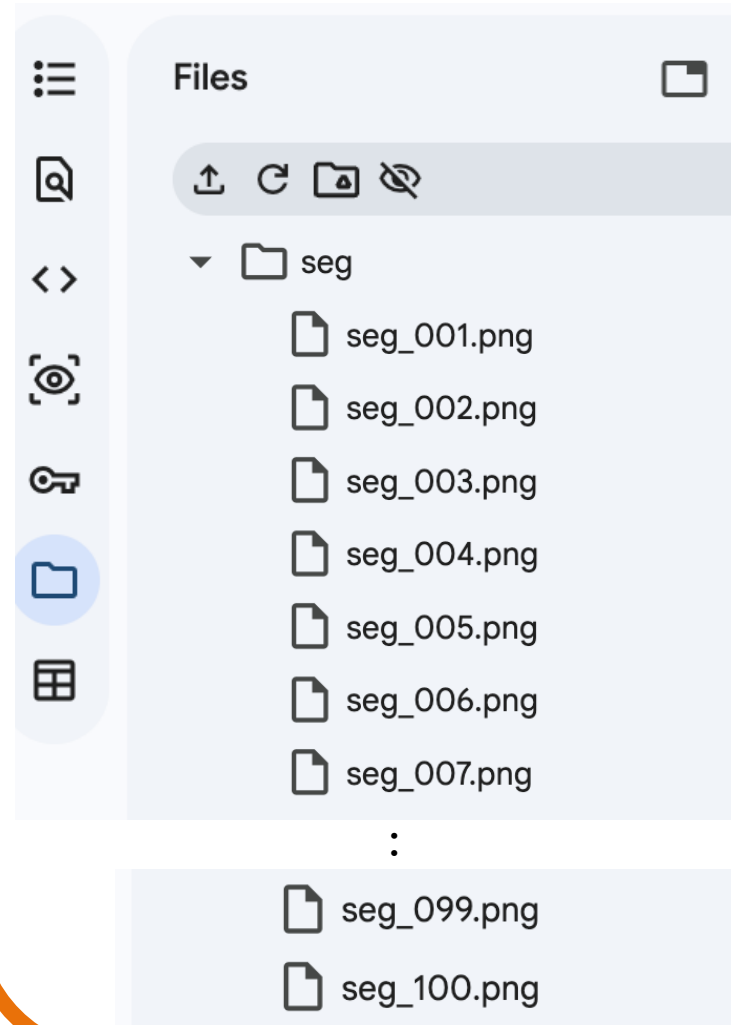
(8 puntos)

3.1 > (1 punto) Crear el directorio 'seg' donde

3.2 > (5 puntos) Segmentar con un umbral a
debe crear una imagen seg_nnn.png que d

3.3 > (1 punto) Desplegar la imagen 'seg/se

3.4 > (1 punto) Desplegar la imagen 'seg/se



se

PREGUNTA 04: DEFINIR FUNCIONES DE EXTRACCIÓN

(8 puntos)

4.1 > (1 punto) Definir la función de área normalizada (función: "a = NormalizedArea(S)", donde 'S' es la imagen segmentada y 'a' el área normalizada.

4.2 > (1 punto) Desplegar las imágenes "seg_043.png" y "seg_009.png" e imprimir el área normalizada para cada una de ellas.

4.3 > (5 puntos) Basándose en el "número de Euler" definir la función de número de agujeros (función: "Na = HolesNumber(S)", donde 'S' es la imagen segmentada y 'Na' el número de agujeros. **ATENCIÓN: Para esta función utilice IA con algún prompt como el mostrado en clases. DEBE incluir en los comentarios del código el prompt empleado.**

4.4 > (1 punto) Desplegar las imágenes "seg_043.png" y "seg_009.png" e imprimir el número de agujeros para cada una de ellas.

PREGUNTA 04: DEFINIR FU

(8 puntos)

4.1 > (1 punto) Definir la función de área normalizada.

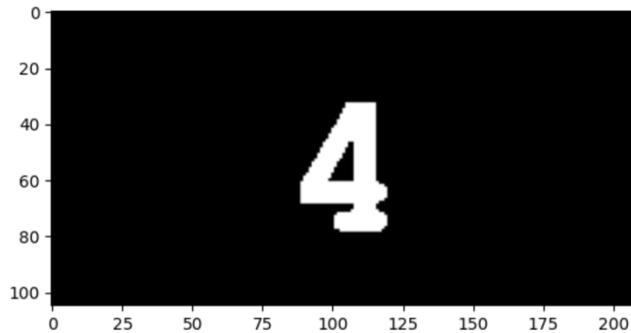
4.2 > (1 punto) Desplegar las imágenes "seg"

4.3 > (5 puntos) Basándose en el "número de clases. **DEBE** incluir en los comentarios del

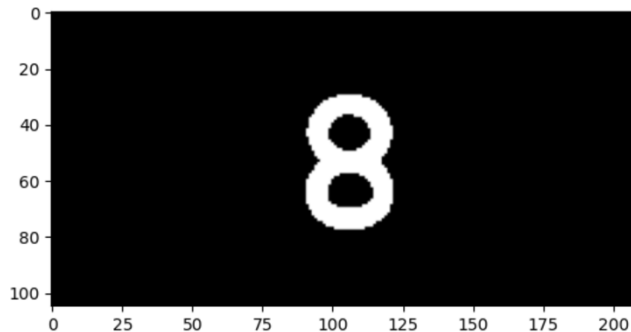
4.4 > (1 punto) Desplegar las imágenes "seg"

```
def NormalizedArea(S,echo='off'):  
def HolesNumber(S,echo='off'):
```

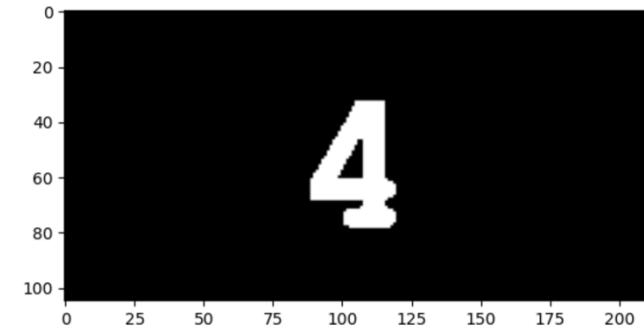
normalized area = 0.918141592920354



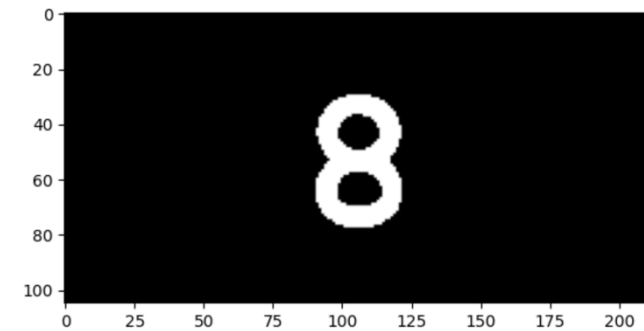
normalized area = 0.7547619047619047



... holes = 1



holes = 2



PREGUNTA 05: EXTRAER CARACTERÍSTICAS

5.1 > (6 puntos) Definir una matriz "X" de 100 x 2 elementos, donde la fila i (para $i = 0, 1, \dots, 99$) está definida como:

- $X[i, 0]$ = área normalizada del dígito de la subimagen i
- $X[i, 1]$ = número de agujeros del dígito de la subimagen i

PREGUNTA 05: EXTRA

5.1 > (6 puntos) Definir una matriz "X"

- $X[i,0]$ = área normalizada del c
- $X[i,1]$ = número de agujeros de

X	Área normalizada	Número de Agujeros
array([[0.64126984, 1.		]
	[1.	, 0.]
	[1.	, 0.]
	[1.	, 0.]
	[0.90807175, 1.	]
	[1.	, 0.]
	[0.83545544, 1.	]
	[1.	, 0.]
	[0.7547619 , 2.	]
	[0.81408451, 1.	]
	[1.	, 0.]
	[1.	, 0.]
	[1.	, 0.]
	[1.	, 0.]
	[0.81683626, 1.	]
	[0.64589905, 1.	]
	[1.	, 0.]
	[0.91415831, 1.	]

ATENCIÓN: EJECUTE ESTE CÓDIGO

Este código corresponde al vector de la clasificación ideal (conocido como el "Ground Truth") para cada una de las 100 subimágenes. De esta manera:

- $Y[i]$ es '1' si la subimagen i es un ocho.
- $Y[i]$ es '0' si la subimagen i NO es un ocho.

para $i = 0, 1, \dots, 99$.

Además se definen dos variables adicionales:

- $X1$: filas de X de los ochos (matriz de 10×2)
- $X0$: filas de X de los no-ochos (matriz de 90×2)

```
Y = np.array([0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
              0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
              1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
              0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
              0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
              0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1])
```

```
# The classes are two: class 0 (not-eights) and class 1 (eights)
```

```
X0 = X[Y==0,:] # not-eights
X1 = X[Y==1,:] # eights
```

PREGUNTA 06: ANÁLISIS DE CARACTERÍSTICAS

(4 puntos)

6.1 > (1 punto) Grafique el histograma para el área normalizada

6.2 > (1 punto) Grafique el histograma para el número de agujeros

6.3 > (1 punto) Copia en la variable Xs la columna 0 (o bien la columna 1) de X, si es que consideras (de manera visual, "al ojo") que el área normalizada (o bien el número de agujeros) separa mejor las dos clases.

6.4 > (1 punto) Defina la variable de umbral 'thc' de manera manual ("al ojo") que sirve para separar en Xs las dos clases.

PREGUNTA 06: ANÁLISIS D

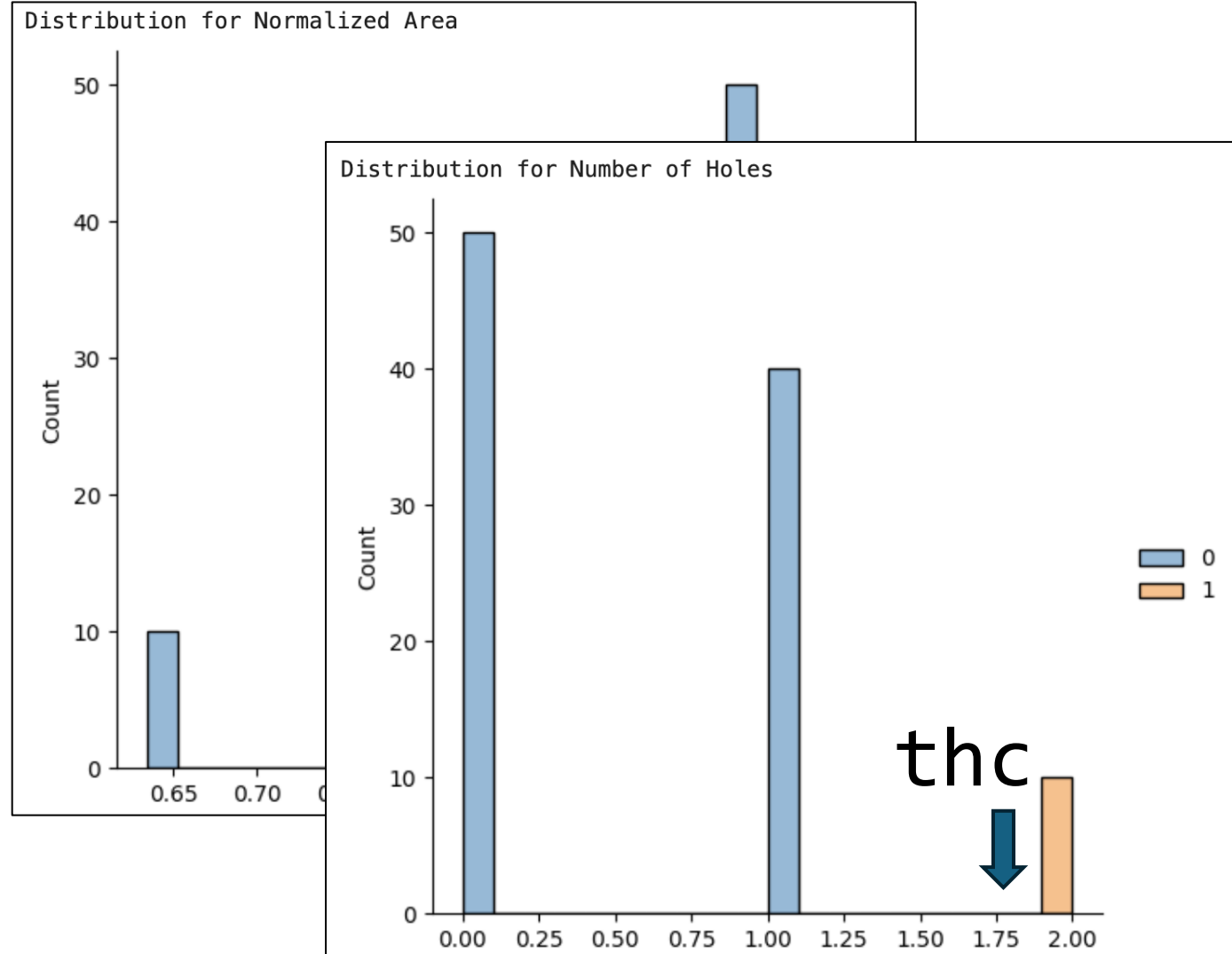
(4 puntos)

6.1 > (1 punto) Grafique el histograma para

6.2 > (1 punto) Grafique el histograma para

6.3 > (1 punto) Copia en la variable Xs la col
normalizada (o bien el número de agujeros)

6.4 > (1 punto) Defina la variable de umbral



PREGUNTA 07: CLASIFICACIÓN

(2 puntos)

7.1 > Defina el vector Y_c de 100 elementos (de números enteros!), donde

- $Y_c[i]$ es '1' si la subimagen i es clasificada como ocho.
- $Y_c[i]$ es '0' si la subimagen i es clasificada como no-ocho.

PREGUNTA 07

(2 puntos)

7.1 > Defina el vector Yc

- Yc[i] es '1' si la su
- Yc[i] es '0' si la s

Yc

```
array([0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1], dtype=uint8)
```

Los '0' indican que el dígito ha sido clasificado como no-ocho, mientras que los '1' indican que ha sido clasificado como ocho

ATENCIÓN: EJECUTE ESTE CÓDIGO

Este código define dos variables adicionales:

- Y0: elementos de Yc clasificados como no-ochos (deberían ser 90 ceros)
- Y1: elementos de Yc clasificados como ochos (deberían ser 10 unos)

- ▶ # Clasificación in Testing subset

```
Y0 = Yc[Y==0] # Classification of 0
```

```
Y1 = Yc[Y==1]    # Classification of 1
```

```
print('Muestras de no-ochos clasificadas como no-ochos: ', Y0)
```

```
print('Muestras de ochos clasificadas como ochos: ',Y1)
```

```
... Muestras de no-ochos clasificadas como no-ochos: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
    0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
Muestras de ochos      clasificadas como      ochos: [1 1 1 1 1 1 1 1 1 1]
```

PREGUNTA 08: EVALUACIÓN

(3 puntos)

8.1 > (1 punto) Defina la matriz de confusión (MC)

8.2 > (1 punto) Calcule el Accuracy (Acc)

8.3 > (1 punto) Grafique la matriz de confusión (MC)

PREGUNTA 0

(3 puntos)

8.1 > (1 punto) Defina

8.2 > (1 punto) Calcule

8.3 > (1 punto) Grafique

Testing Confusion Matrix:
[90. 0.]
[0. 10.]

Testing Accuracy = 100.0%

